

Security of Microservices Architectures

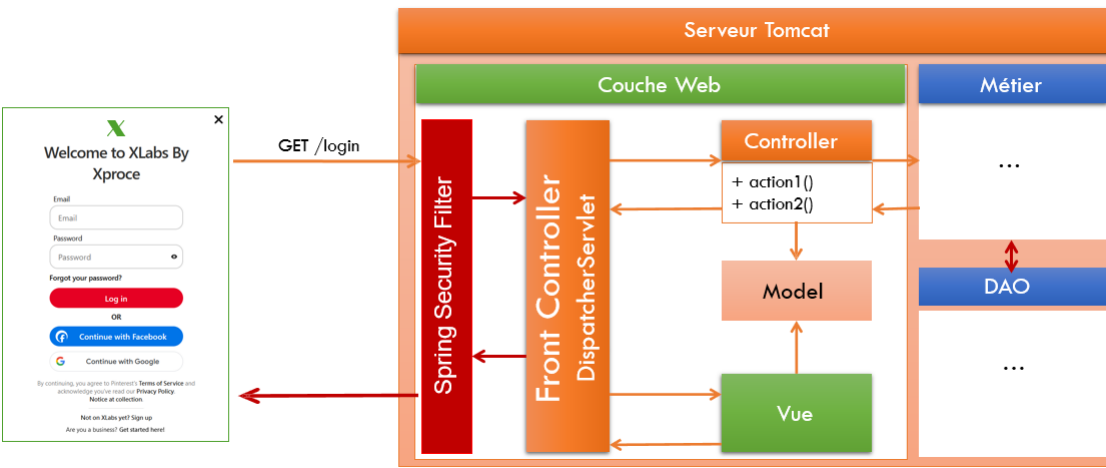
Spring Security, JWT, OAuth2, OIDC, Keycloak ...

Badr Hirchoua
hirchoua.badr@gmail.com





25/10/2025



Spring Security : Concepts de base

Badr Hirchoua
hirchoua.badr@gmail.com

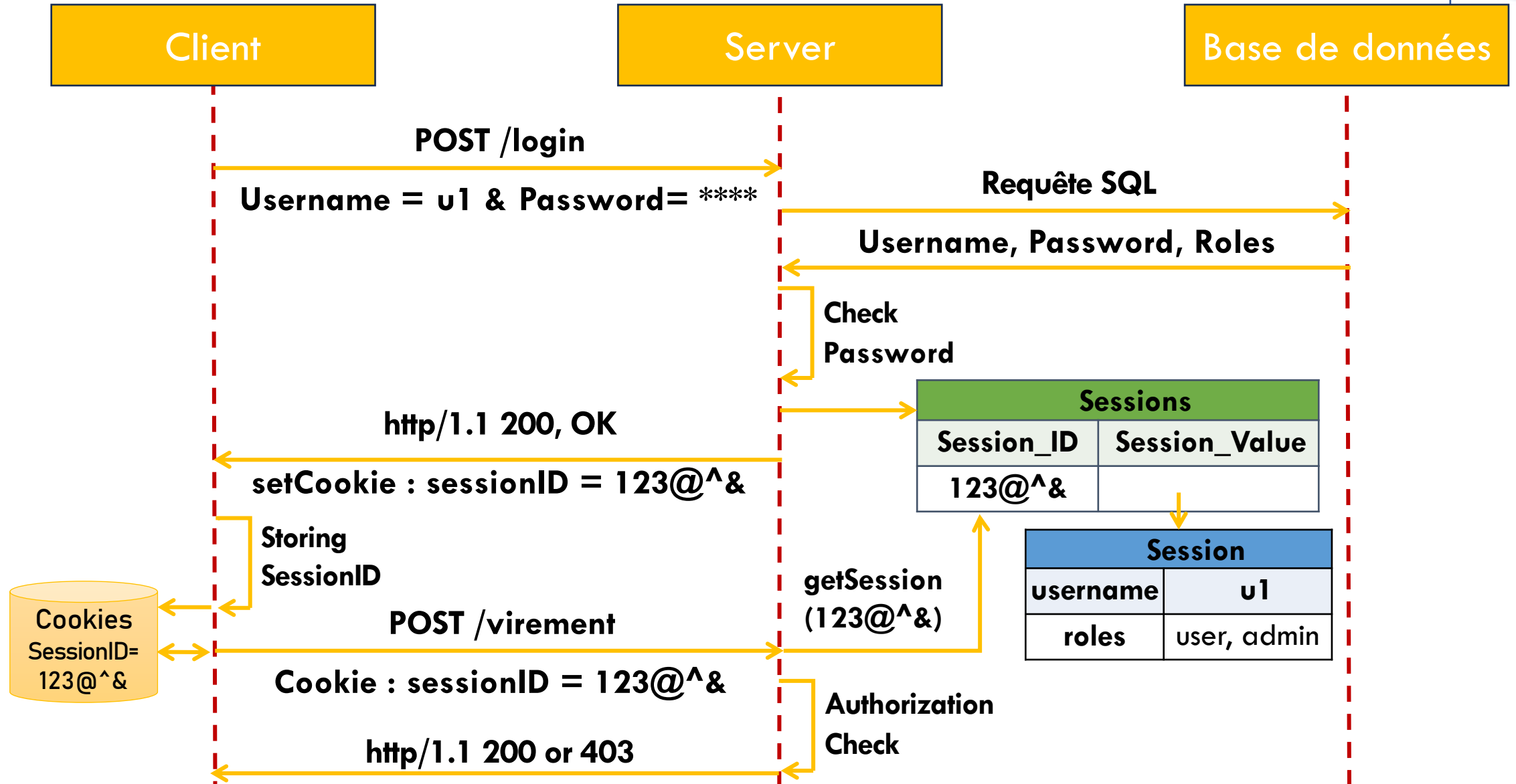




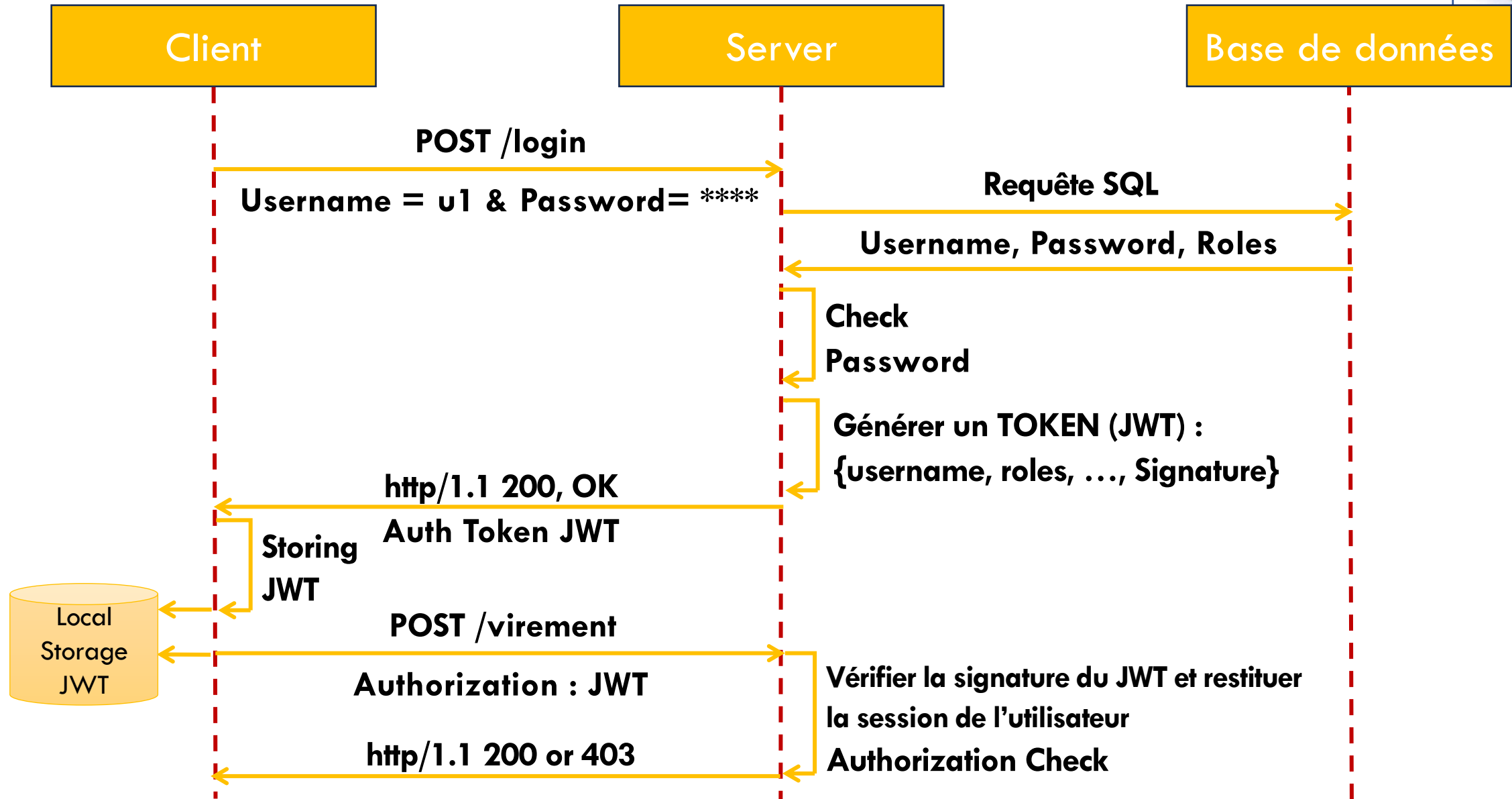
Il existe deux types de modèles d'authentification :

1. **Stateful** (avec état) : Dans ce modèle, les données de la session sont enregistrées côté serveur d'authentification. Cela signifie que le serveur conserve des informations sur l'état de la session de l'utilisateur.
2. **Stateless** (sans état) : Dans ce modèle, les données de la session sont enregistrées dans un jeton (token) d'authentification délivré au client. Contrairement au modèle stateful, le serveur ne stocke pas d'informations sur l'état de la session. Le client doit inclure ce jeton dans chaque requête pour prouver son identité.

Authentication Statful



Authentication Stateless

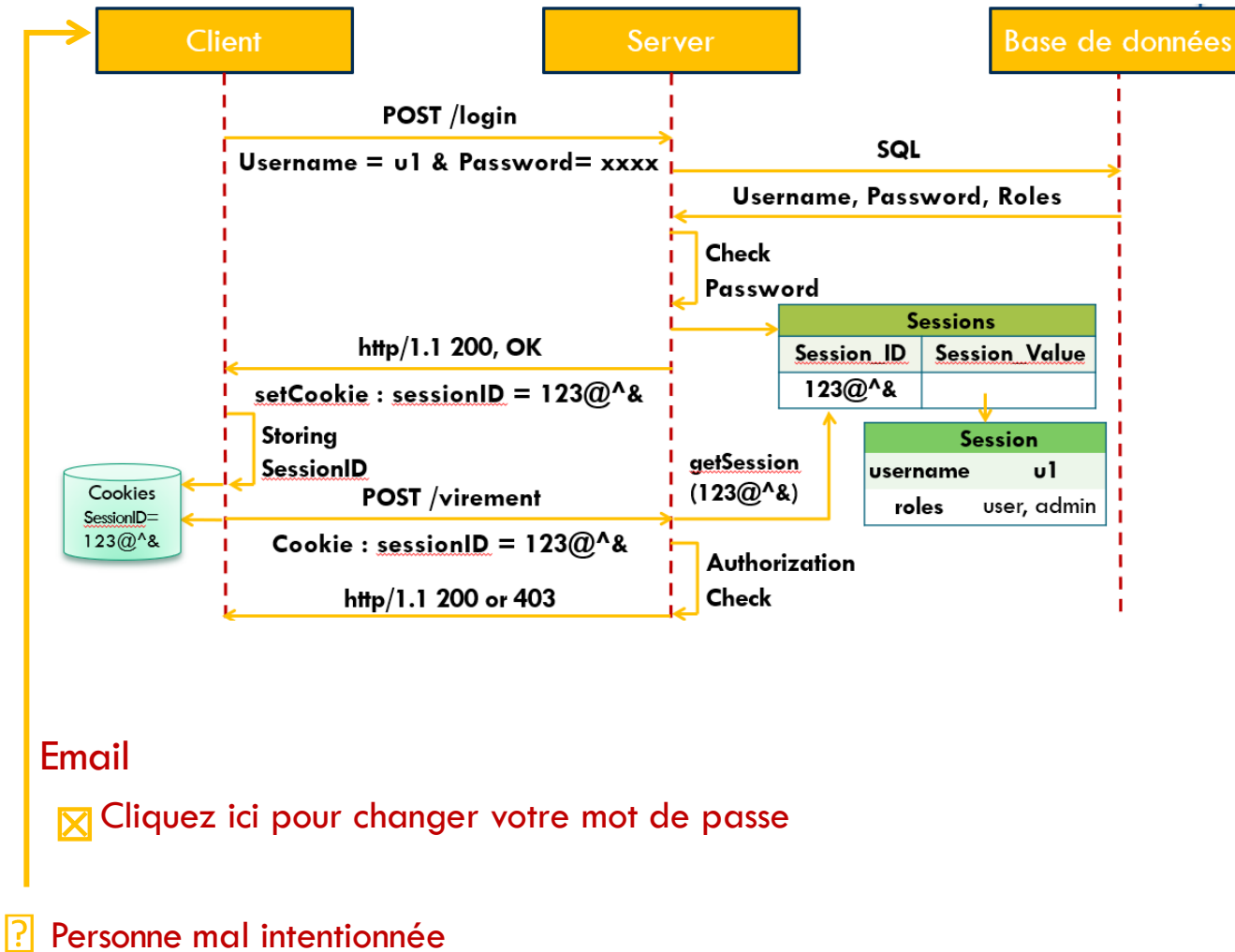


Quelques failles de sécurité : Cross Site Request Forgery (CSRF)



- En sécurité informatique, le Cross-Site Request Forgery (CSRF) est un type de vulnérabilité touchant les services d'authentification web. Voici les points clés à retenir :

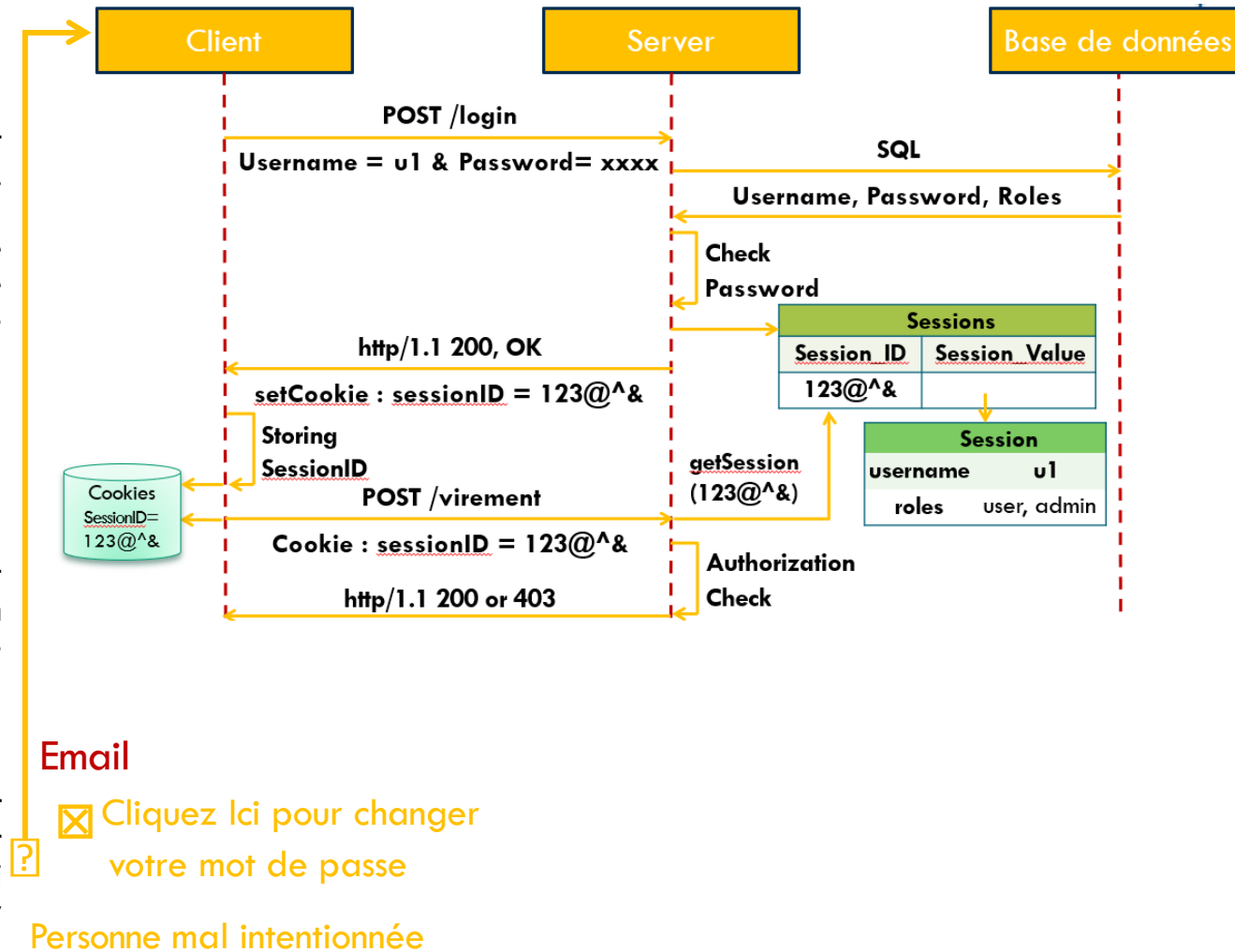
- Le CSRF vise à tromper un utilisateur authentifié en lui faisant exécuter une requête HTTP falsifiée.
- Cette requête pointe vers une action interne au site (par exemple, changer le mot de passe, effectuer un virement).
- L'utilisateur agit sans en avoir conscience et en utilisant ses propres droits.
- L'attaque se déclenche par l'utilisateur lui-même, contournant ainsi de nombreux systèmes d'authentification.



Quelques failles de sécurité : Cross Site Request Forgery (CSRF)



- Badr détient un compte bancaire qui lui permet, entre autres, d'effectuer des virements en ligne depuis son compte vers d'autres comptes bancaires. L'application de la banque utilise un système d'authentification basé exclusivement sur les sessions, avec les identifiants de session (SessionID) stockés dans les cookies.
- Amina possède également un compte dans la même banque et connaît bien la structure du formulaire permettant d'effectuer des virements. Elle envoie à Badr un e-mail contenant un lien hypertexte, l'invitant à cliquer pour découvrir son cadeau d'anniversaire. Cependant, ce message contient également un formulaire invisible destiné à soumettre les données pour effectuer un virement. Parmi les champs cachés de ce formulaire, on trouve notamment :
 - Le montant du virement à effectuer.
 - Le numéro de compte de Amina.
- Lorsque Badr reçoit ce message, sa session bancaire est déjà ouverte, et son identifiant de session (Session ID) est présent dans les cookies. En cliquant sur le lien, Badr vient d'effectuer un virement depuis son compte vers celui de Amina, sans même s'en rendre compte.
- En effet, en cliquant sur le lien, les données du formulaire caché sont transmises au script côté serveur approprié. Comme les cookies sont systématiquement envoyés au serveur du même domaine, celui-ci autorise l'opération, considérant qu'elle provient d'un utilisateur correctement authentifié avec une session valide.





- L'utilisation de jetons de validité (CSRF Synchronizer Token) dans les formulaires est une pratique essentielle pour renforcer la sécurité des applications web. Voici quelques points à considérer :
 1. Validation temporelle: Lorsqu'un formulaire est soumis, assurez-vous qu'il n'est accepté que s'il a été produit récemment (par exemple, dans les dernières minutes). Le jeton de validité (CSRF token) joue un rôle crucial dans cette vérification.
 2. Transmission du jeton: Le jeton de validité doit être inclus dans chaque requête, généralement en tant que paramètre. Un champ caché (de type Hidden) dans le formulaire est souvent utilisé pour stocker ce jeton. Côté serveur, il est essentiel de vérifier ce jeton pour s'assurer que la requête provient d'une source légitime.
 3. Éviter les requêtes HTTP GET pour les actions critiques : Les requêtes HTTP GET sont plus vulnérables aux attaques basées sur les liens hypertexte. Pour les actions de modification de données (ajout, mise à jour, suppression), privilégiez les requêtes HTTP POST. Cela réduit le risque d'attaques simples.



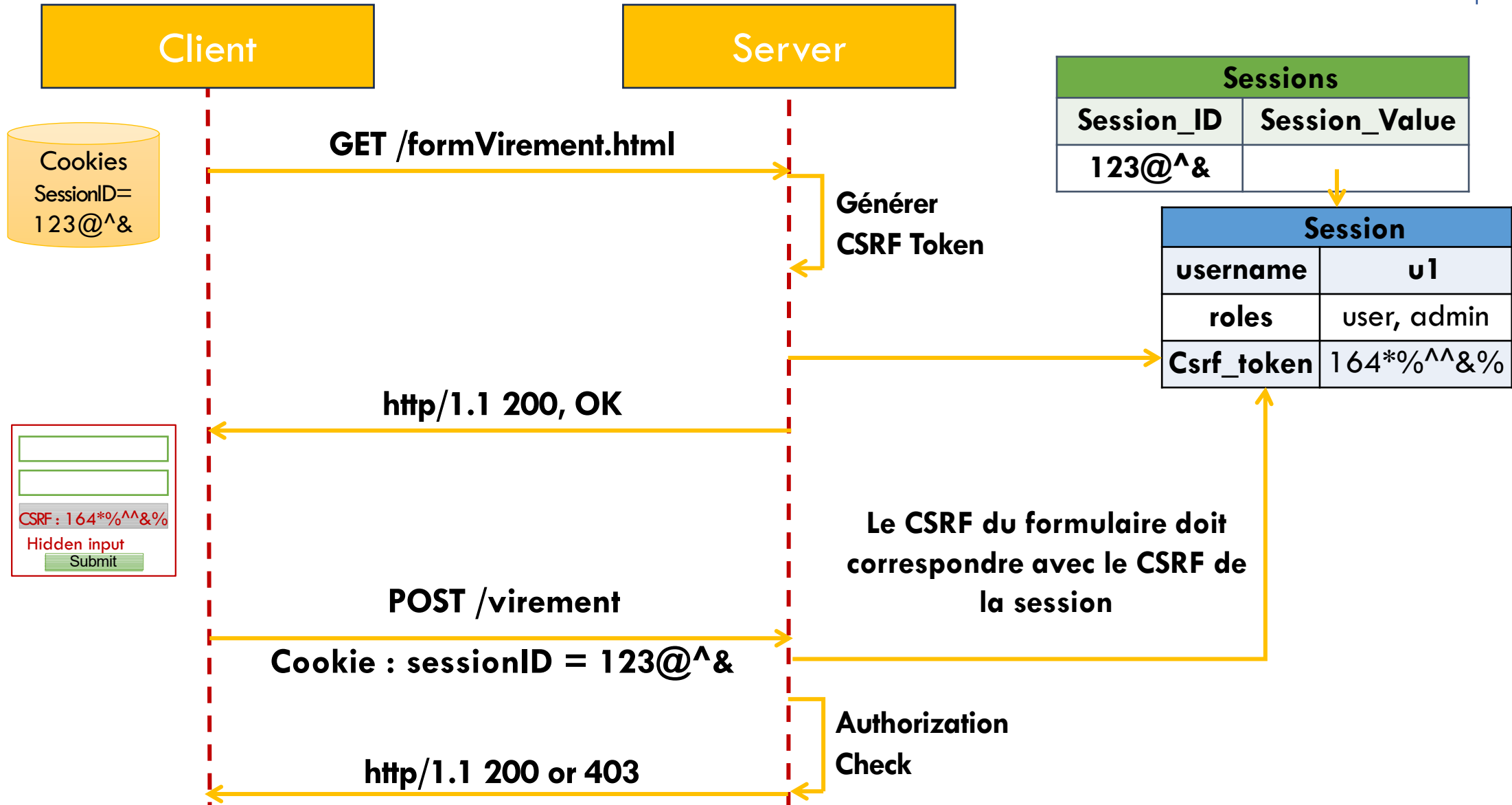
4. Demande de confirmation pour les actions critiques: Bien que cela puisse alourdir l'expérience utilisateur, demander une confirmation avant d'exécuter des actions critiques (par exemple, supprimer un compte) est une bonne pratique. Cela évite les erreurs accidentelles.
5. Confirmation de l'ancien mot de passe : Lorsque l'utilisateur souhaite changer son mot de passe, demandez une confirmation de l'ancien mot de passe. Vous pouvez également utiliser une confirmation par email ou par SMS pour renforcer la sécurité.
6. Vérification du référent (Referrer): Dans les pages sensibles (par exemple, lors de la modification des informations personnelles), vérifiez la provenance du client. Bloquez la requête si la valeur du référent est différente de la page d'où elle devrait théoriquement provenir. Cela prévient les attaques de type CSRF.

Préventions contre les attaques CSRF

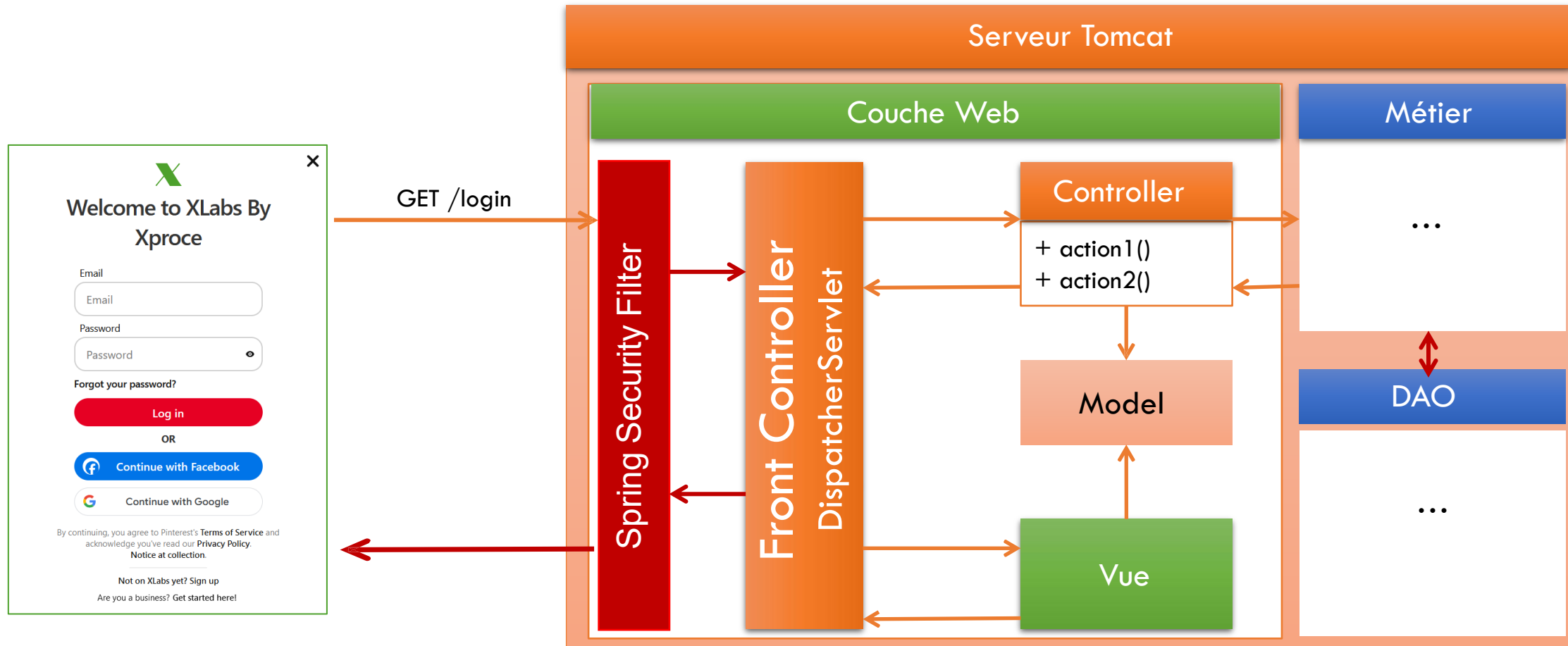


- Utiliser des jetons de validité (CSRF Synchronizer Token) dans les formulaires :
- Faire en sorte qu'un formulaire posté ne soit accepté que s'il a été produit quelques minutes auparavant. Le jeton de validité en sera la preuve.
- Le jeton de validité doit être transmis souvent en paramètre (Dans un champ de type Hidden du formulaire) et vérifié côté serveur.
- Autres :
 - Éviter d'utiliser des requêtes HTTP GET pour effectuer des actions critiques de modification des données (Ajout, Mise à jour, Suppression) : cette technique va naturellement éliminer des attaques simples basées sur les liens hypertexte, mais laissera passer les attaques fondées sur JavaScript, lesquelles sont capables très simplement de lancer des requêtes HTTP POST.
 - Demander des confirmations à l'utilisateur pour les actions critiques, au risque d'alourdir l'enchaînement des formulaires.
 - Demander une confirmation de l'ancien mot de passe à l'utilisateur pour changer celui-ci ou utiliser une confirmation par email ou par SMS.
 - Effectuer une vérification du référent dans les pages sensibles : connaître la provenance du client permet de sécuriser ce genre d'attaques. Ceci consiste à bloquer la requête du client si la valeur de son référent est différente de la page d'où il doit théoriquement provenir.

Authentication Statful



Spring MVC - Security





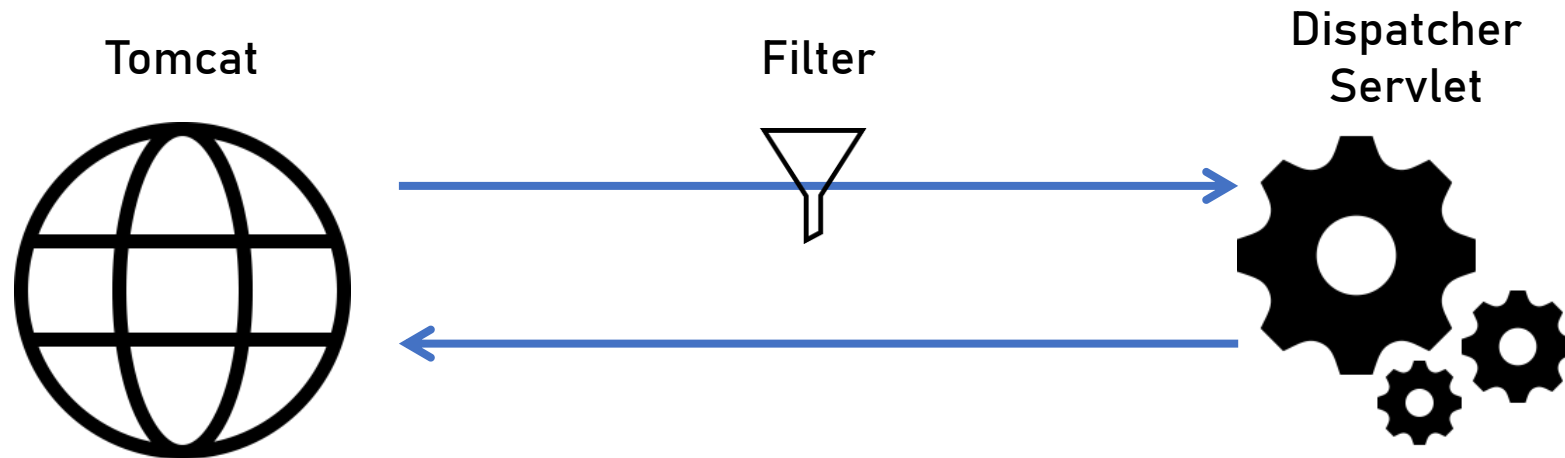
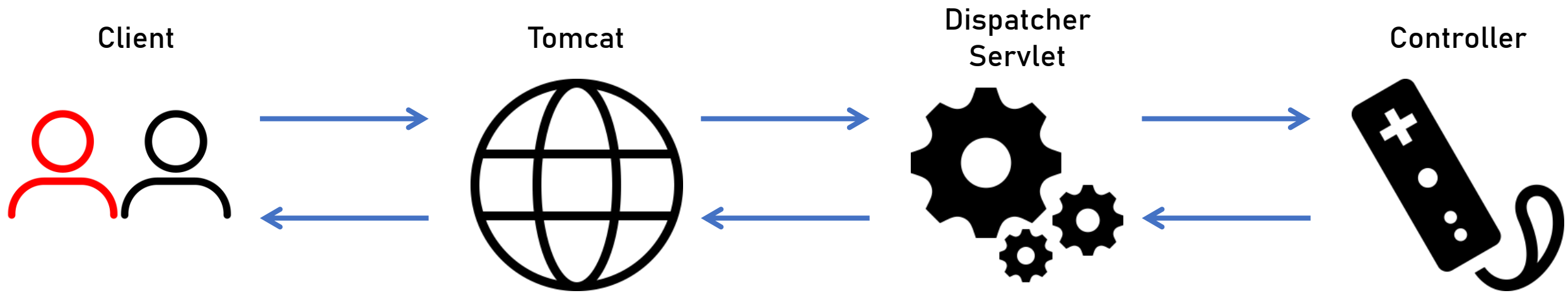
- Spring Security est un module de Spring qui permet de sécuriser les applications Web.
- Spring Security configure des filtres (springSecurityFilterChain) qui permet d'intercepter les requêtes HTTP et de vérifier si l'utilisateur authentifié dispose des droits d'accès à la ressource demandée.
- Les actions du contrôleur ne seront invoquées que si l'utilisateur authentifié dispose de l'un des rôles attribués à l'action.
- Spring Security peut configurer les rôles associés aux utilisateurs en utilisant différentes solution (Mémoire, Base de Données, etc..)

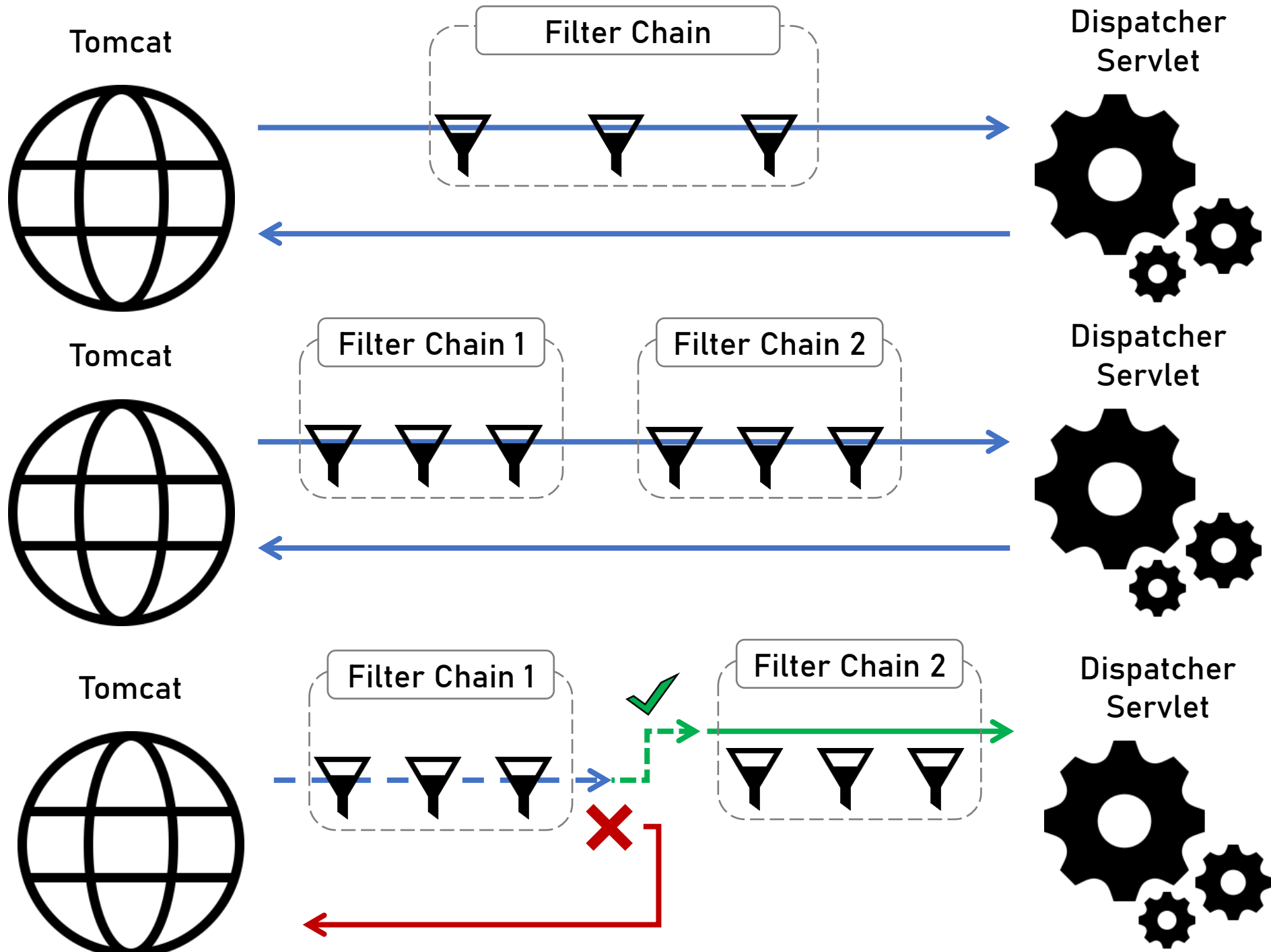


Spring Security Internal Workflow

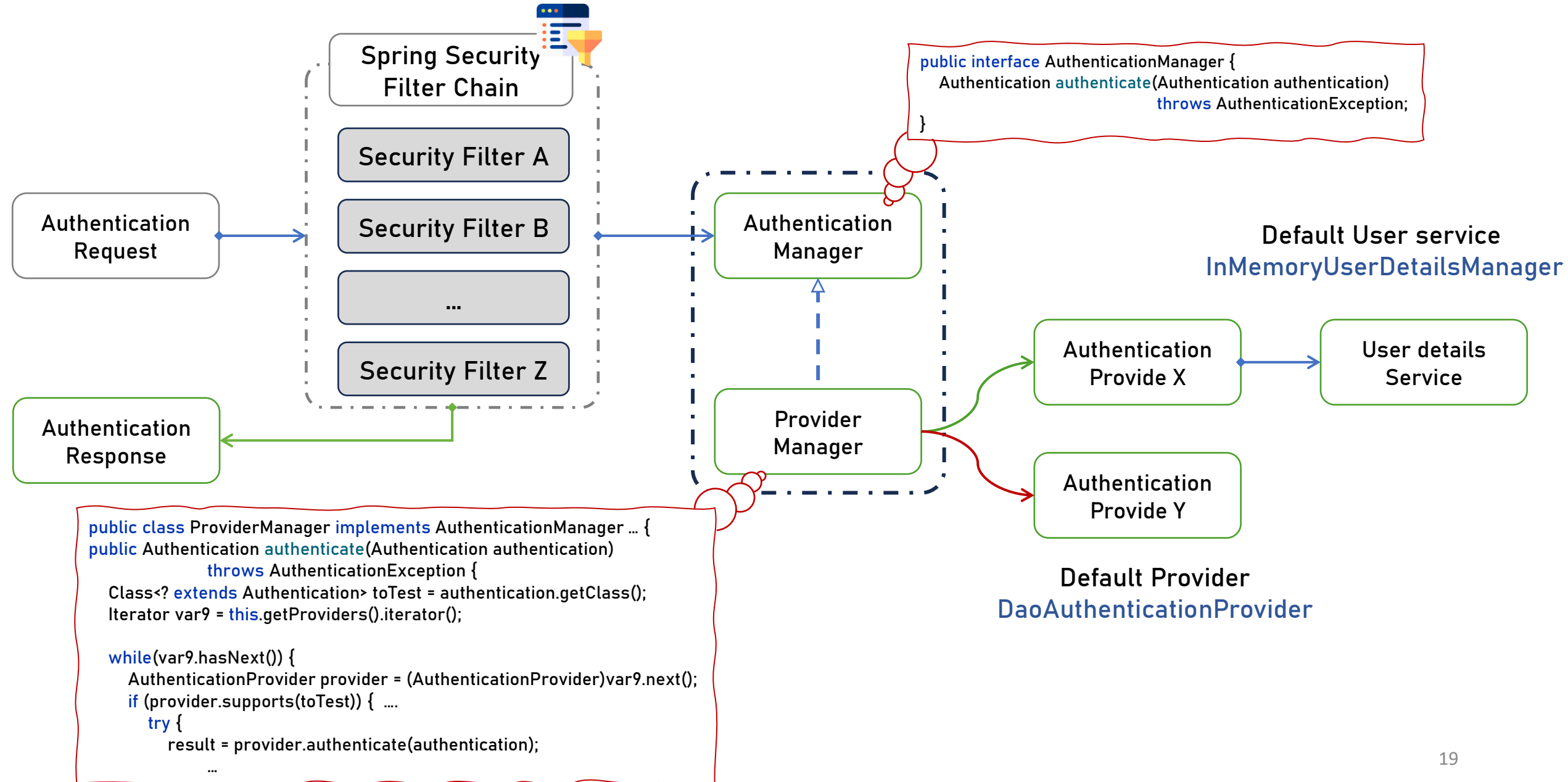
Badr Hirchoua
hirchoua.badr@gmail.com







Default Filter UsernamePasswordAuthenticationFilter





Welcome to XLabs By Xproce

Email

Password

Forgot your password?

Log in

OR

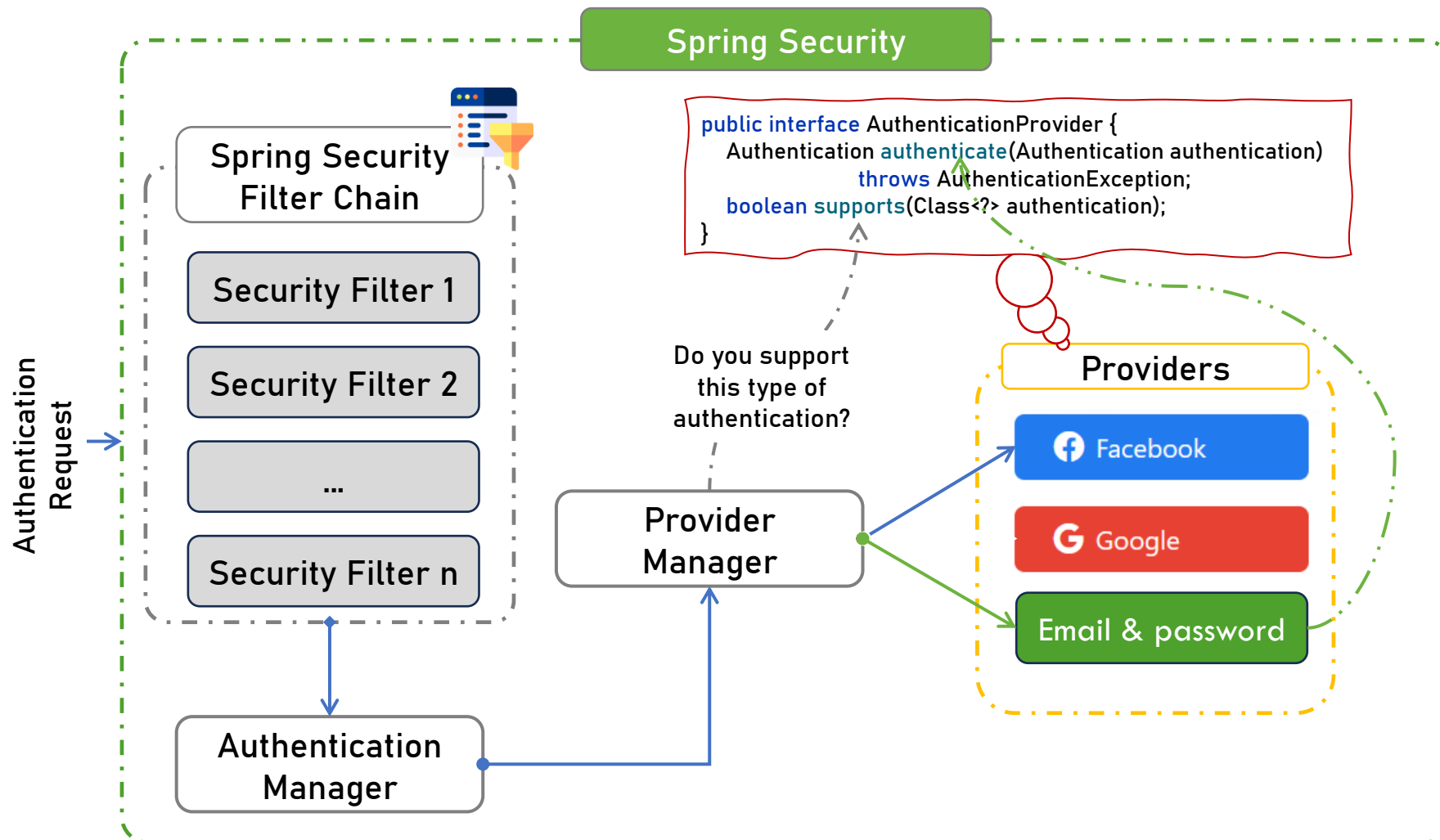
 Continue with Facebook

 Continue with Google

By continuing, you agree to Pinterest's [Terms of Service](#) and acknowledge you've read our [Privacy Policy](#).
Notice at collection.

Not on XLabs yet? Sign up

Are you a business? Get started here!



@Configuration

@EnableWebSecurity

```
public class SecurityConfig {
```

@Bean

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```
    http
```

```
        .csrf(Customizer.withDefaults())
```

```
        .httpBasic(Customizer.withDefaults())
```

```
        .formLogin(Customizer.withDefaults())
```

```
        .authorizeHttpRequests(authorize -> authorize  
            .anyRequest().authenticated()  
        );
```

```
    return http.build();
```

```
}
```

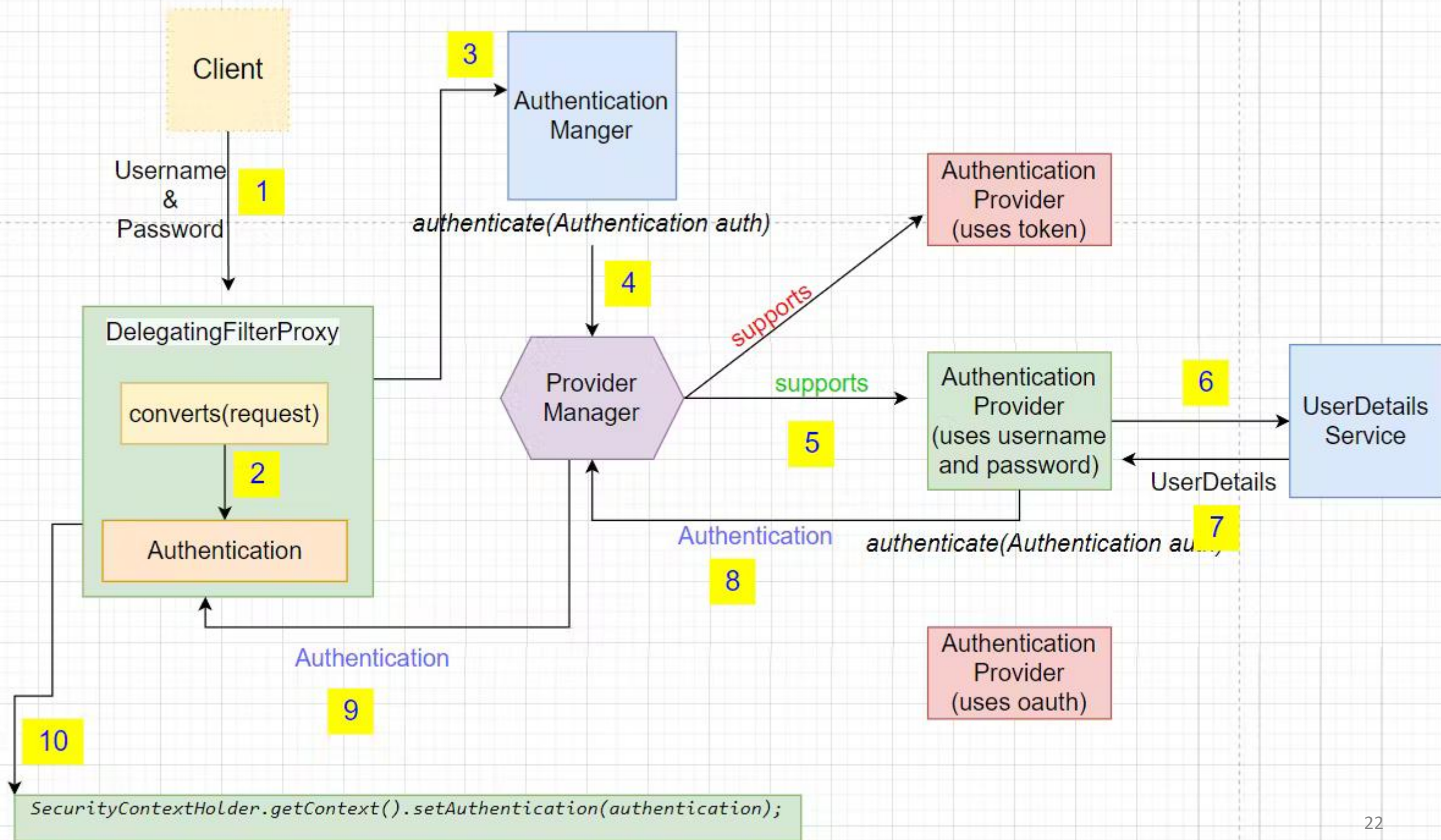
```
}
```

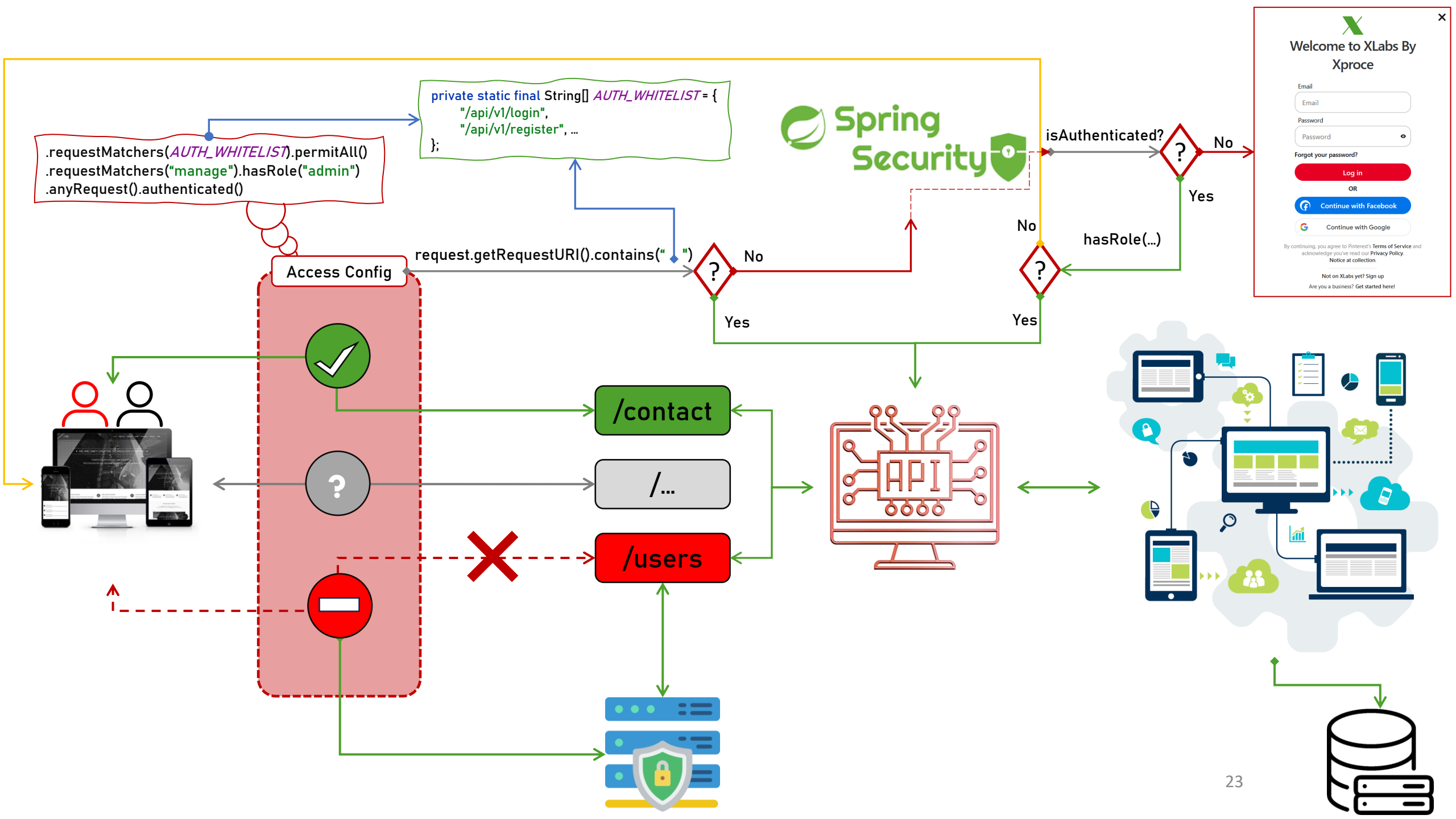
1. First, the `CsrfFilter` is invoked to protect against [CSRF attacks](#).

2. Second, [the authentication filters](#) are invoked to authenticate the request.

3. Third, [the AuthorizationFilter](#) is invoked to authorize the request.

Filter	Added by
CsrfFilter	HttpSecurity#csrf
UsernamePasswordAuthenticationFilter	HttpSecurity#formLogin
BasicAuthenticationFilter	HttpSecurity#httpBasic
AuthorizationFilter	HttpSecurity#authorizeHttpRequests







Welcome to XLabs By Xproce

Email

Password

Forgot your password?

Log in

OR

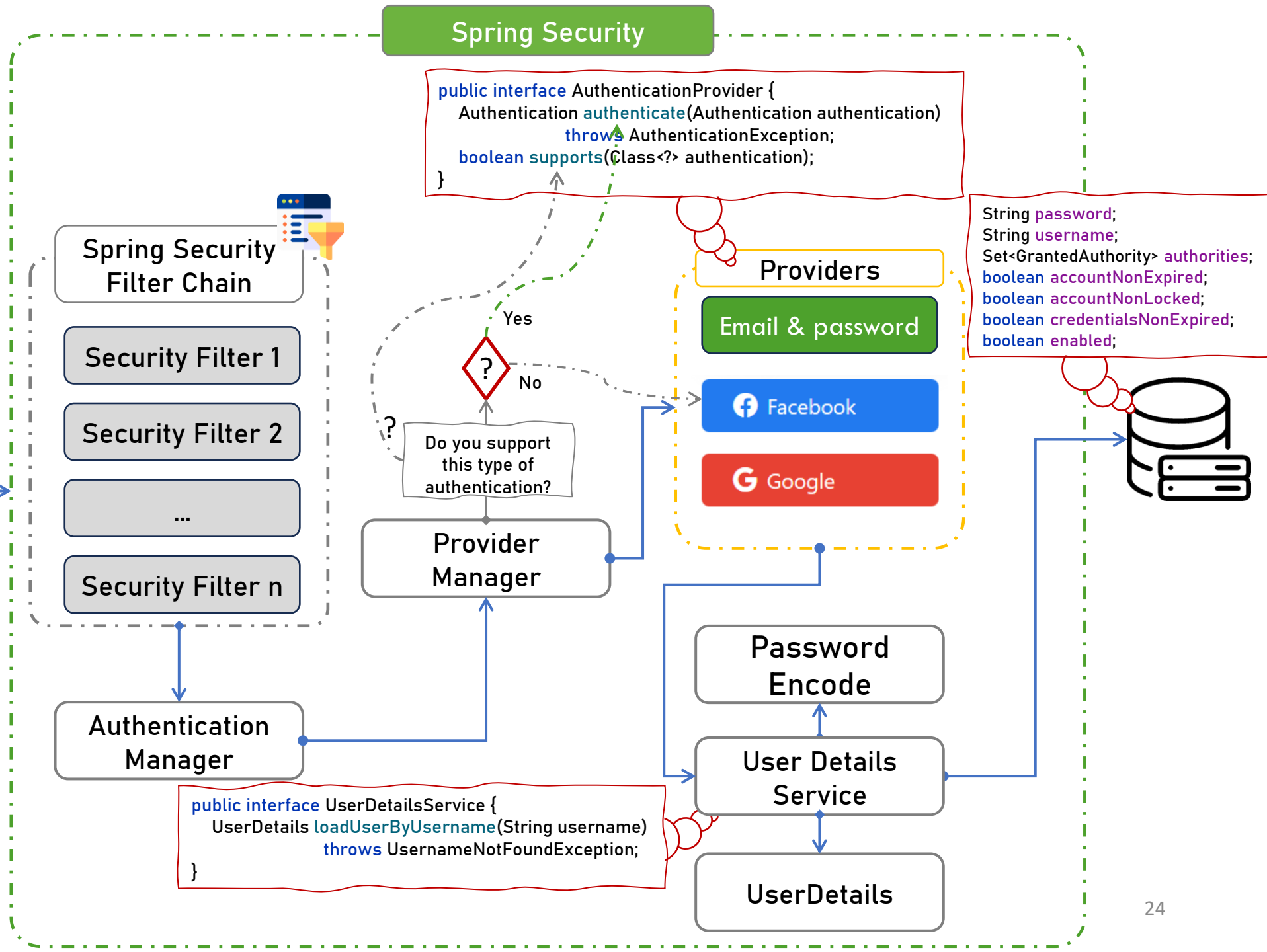
 Continue with Facebook

 Continue with Google

By continuing, you agree to Pinterest's [Terms of Service](#) and acknowledge you've read our [Privacy Policy](#).
Notice at collection.

Not on XLabs yet? [Sign up](#)
Are you a business? [Get started here!](#)

Authentication Request



Spring Security

User

```
public class User implements UserDetails, CredentialsContainer {
    private static final long serialVersionUID = 620L;
    private static final Log logger = LoggerFactory.getLog(User.class);
    private String password;
    private final String username;
    private final Set<GrantedAuthority> authorities;
    private final boolean accountNonExpired;
    private final boolean accountNonLocked;
    private final boolean credentialsNonExpired;
    private final boolean enabled;
    ...
}
```

```
public interface UserDetails extends Serializable {
    Collection<? extends GrantedAuthority> getAuthorities();
    String getPassword();
    String getUsername();
    default boolean isAccountNonExpired() { return true; }
    default boolean isAccountNonLocked() { return true; }
    default boolean isCredentialsNonExpired() { return true; }
    default boolean isEnabled() { return true; }
}
```

UserDetails

```
String password;
String username;
Set<GrantedAuthority> authorities;
boolean accountNonExpired;
boolean accountNonLocked;
boolean credentialsNonExpired;
boolean enabled;
```

User Details Service

```
public interface UserDetailsService {
    UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException;
}
```

User Details Manager

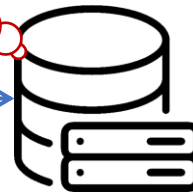
```
public interface UserDetailsManager extends UserDetailsService {
    void createUser(UserDetails user);
    void updateUser(UserDetails user);
    void deleteUser(String username);
    void changePassword(String oldPassword, String newPassword);
    boolean userExists(String username);
}
```

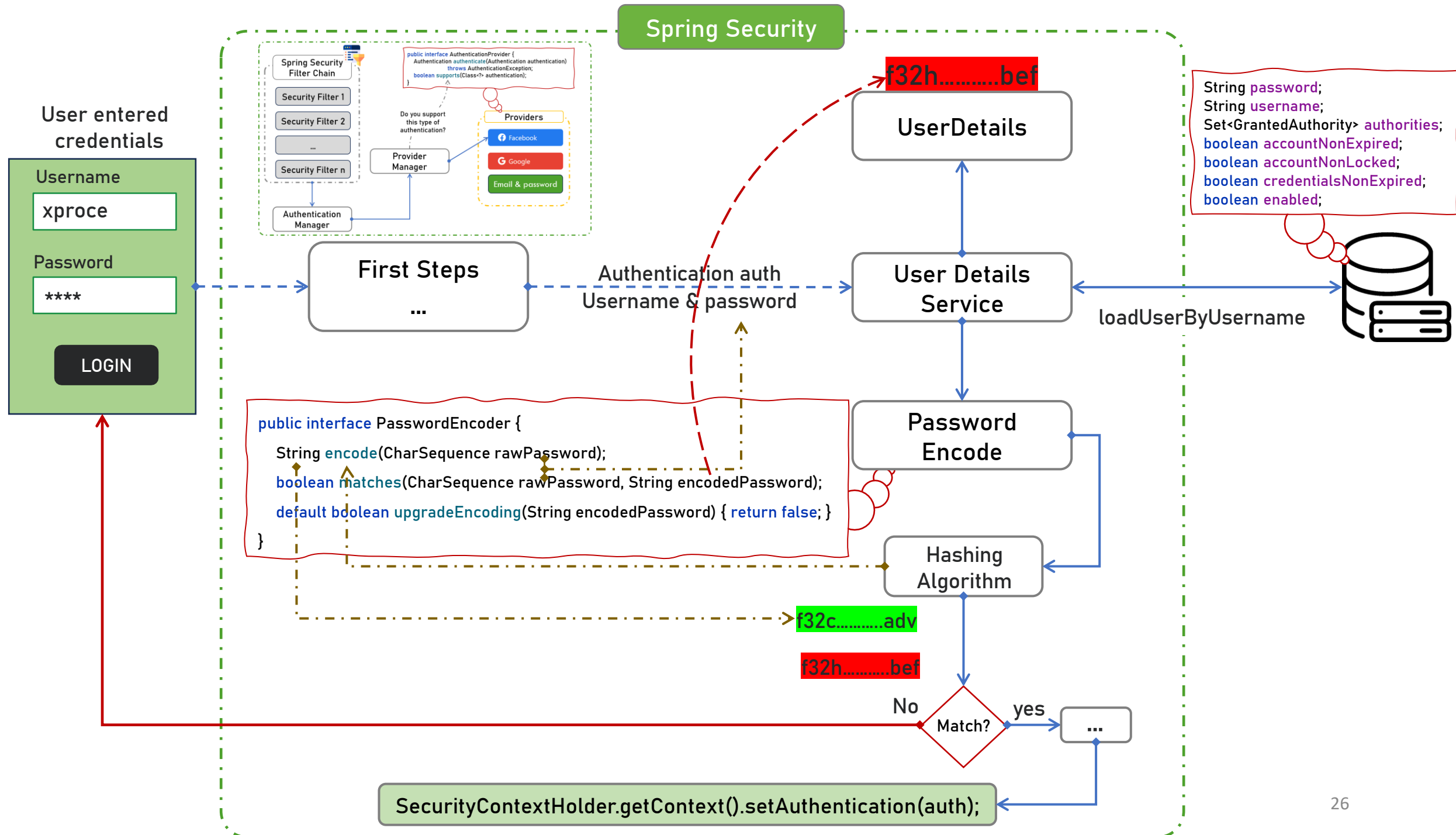
User

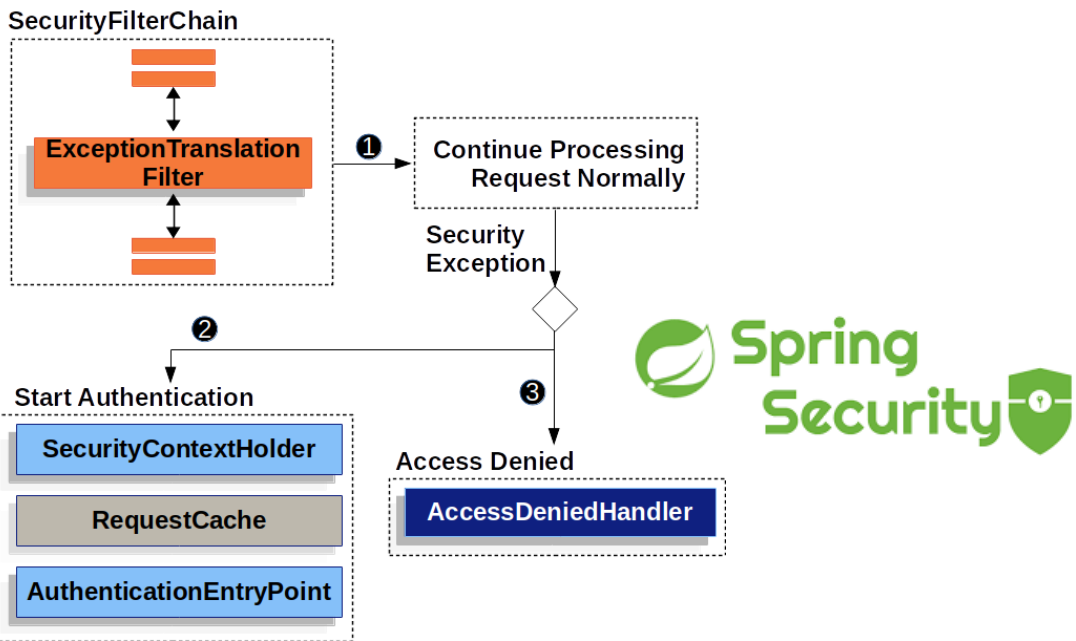
UserDetails

User Details Service

User Details Manager







Filters



Badr Hirchoua

hirchoua.badr@gmail.com

